

NEFCLASS-X — a soft computing tool to build readable fuzzy classifiers

D Nauck and R Kruse

Neuro-fuzzy classification systems offer a means of obtaining fuzzy classification rules by a learning algorithm. Although it is usually no problem to find a suitable fuzzy classifier by learning from data, it can, however, be hard to obtain a classifier that can be interpreted conveniently. There is usually a trade-off between accuracy and readability. This paper discusses NEFCLASS — a neuro-fuzzy approach for classification problems — and its implementation NEFCLASS-X. It is shown how a readable fuzzy classifier can be obtained by a learning process and how interactive strategies for pruning rules and variables from a trained classifier can enhance its interpretability.

1. Introduction

The term ‘soft computing’ was coined by the father of fuzzy logic, Lotfi A Zadeh [1]. In contrast to hard computing that is based on using crisp numbers or symbols, the principal aim of soft computing is to exploit the tolerance of uncertainty and vagueness in the area of cognitive reasoning. This can lead to considerable reduction in complexity when real-world problems have to be solved. Soft computing allows the realisation of easy-to-handle, robust, and low-priced solutions in many application areas. In addition soft computing solutions based on fuzzy systems, for instance, are often much simpler to understand and to maintain.

The notion of soft computing subsumes several approaches like fuzzy systems, neural networks, evolutionary computation and various methods of probabilistic reasoning, and especially the combinations of these methods, such as neuro-fuzzy systems.

This paper discusses one aspect of soft computing — neuro-fuzzy systems. Neuro-fuzzy systems are usually described as combinations of neural networks and fuzzy systems. However, this description is far too ‘fuzzy’. We prefer to think of neuro-fuzzy approaches as a special way to learn fuzzy systems from data. Our interpretation is described by the following five points:

- a neuro-fuzzy system is a fuzzy system trained by a (heuristic) learning algorithm (usually) derived from neural networks,
- a neuro-fuzzy system can be represented by a feed-forward neural network architecture — but this is not a prerequisite to training, it is merely a convenience to visualise the structure and the flow of data,
- a neuro-fuzzy system can always be interpreted in terms of fuzzy if-then rules,
- a neuro-fuzzy system’s training procedure takes the semantics of the underlying fuzzy model into account to preserve the linguistic interpretability of the model,
- a neuro-fuzzy system performs (special cases of) function approximation — it has nothing to do with fuzzy logic in the narrow sense, i.e. generalised logical rules.

Modern neuro-fuzzy systems are often represented as multilayer feedforward neural networks [2—8]. In addition, there are also combinations of fuzzy techniques with other neural network architectures, for example self-organising feature maps [9, 10], or fuzzy associative memories [11]. Some approaches refrain from representing a fuzzy system in a network architecture. They just apply a learning procedure to the parameters of the fuzzy system, or explicitly use a neural network to determine them. There are several different approaches which have much in common, but differ in implementational aspects. For an overview on neuro-fuzzy systems, see for example Buckley and Hayashi [3], Nauck et al [7], and Lin and Lee [12].

This paper discusses neuro-fuzzy modelling in the area of classification. Classification of customer data, for instance, is an important data analysis problem for modern companies. It is, for example, important to know if a customer may be interested in a certain mailing, if certain services should be offered to him, or if there is a danger that he might cancel a contract.

Classical approaches to classification usually use statistical methods like discriminant analysis; but statistical approaches are often based on assumptions that do not hold for real-world data and the results can be hard to interpret. The readability of a solution, however, is often very important for its acceptance and application.

It is also possible to use a fuzzy system for classification. A fuzzy classifier is based on linguistic (fuzzy) rules like, for example:

if income is high **and** age is approximately 40 **and** ...
and residence area is middle class

then customer belongs to class A ,

where the vague expressions like high, approximately 40, etc, are represented by fuzzy sets.

What is the advantage of having another method for classification? A fuzzy classifier is not a replacement for methods like statistics or other forms of machine learning and it does not yield better results. Fuzzy classification offers a different way of achieving the same goal. If a decision is made to use a fuzzy classifier, usually the following advantages are considered:

- vague knowledge can be used,
- the classifier is interpretable in the form of linguistic rules,
- from an applicational point of view, the classifier is easy to implement, to use and to understand.

It is necessary to stress that fuzzy classifiers which use rules like the one given above are only useful if direct dependencies of features to class informations are to be modelled. It is implicitly assumed that the features themselves are independent from each other. If more complex domains need to be modelled, where dependencies or conditional independencies between all variables of a given problem have to be explicitly represented, then graphical models (dependency networks), for instance, Bayesian networks [13, 14] or possibilistic networks [15, 16], should be preferred. Fuzzy classifiers can be viewed as an alternative to neural networks, regression models, or nearest-neighbour classifiers, if the above-mentioned advantages are of interest.

Neuro-fuzzy classification systems offer a means of obtaining fuzzy classification rules by a learning algorithm. It is usually no problem to find a suitable fuzzy classifier by learning from data, but it can be hard to obtain a classifier that can be interpreted conveniently. This paper describes NEFCLASS [17], a neuro-fuzzy approach for classification. NEFCLASS is especially aimed at obtaining interpretable results, because it constrains the learning algorithm and allows the user to interactively influence the training process. The interpretability of the learning result can be improved by interactive strategies for pruning rules and variables from a trained classifier.

2. Fuzzy classification

We consider the task to assign a pattern which is a vector $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$ to a class C which is a subset of $\mathbb{R}^n$. We want to solve this task by learning fuzzy classification rules from a set of labelled training data, where each pattern of the training set is assigned to the class to which it most likely belongs. The fuzzy rules used to classify the data are of the form:

R: if x_1 is μ_1 **and** x_2 is μ_2 **and** ... **and** x_n is μ_n **then** the pattern $(x_1, x_2, \dots, x_n)$ belongs to class C_i , where $\mu_1, \dots, \mu_n$ are fuzzy sets.

Such a rule is evaluated by determining the degree of membership of each feature $x_i \in \mathbb{R}$ to the respective fuzzy set $\mu_i: \mathbb{R} \rightarrow [0, 1]$ in the antecedent of the rule. The degrees of membership are combined by the minimum that is used to implement the operator 'and'. The resulting number from $[0, 1]$ indicates the degree of membership to the class $C_i \subseteq \mathbb{R}^n$ specified in the consequent of the rule. This way a pattern can belong to several classes with different degrees of membership.

The task of the NEFCLASS approach is to discover these rules and to learn the shape of the membership functions in order to determine the correct class or category of a given input pattern. The patterns' feature values are described by linguistic terms (like big, small, etc) which are represented by fuzzy sets. The complete classifier consists of a number of linguistic rules (fuzzy rules).

The main goal of NEFCLASS is to create a readable classifier that also provides an acceptable accuracy. However, the user has to be aware that readability and accuracy usually do not go together. An interpretable fuzzy system should display the following features:

- few meaningful fuzzy rules with few variables in their antecedents,
- few meaningful fuzzy sets for each variable,
- there are no rule weights [18],

- identical linguistic terms are represented by identical fuzzy sets,
- only normal fuzzy sets are used, or, even better, fuzzy numbers or fuzzy intervals [16].

These features pose a lot of restrictions on how a fuzzy system can be created from training data. If high performance of the fuzzy system is the main goal, it is necessary to fit the system to the data very accurately. This approach, however, usually yields fuzzy systems that do not display the above-mentioned features [19] and are in fact black-box models. A user has therefore to decide what is more important — accuracy or readability. NEFCLASS provides the means to ensure the readability of the solution by giving the user complete control over the learning process. It should also be stressed that interpretable solutions can usually not be obtained without the user's co-operation. The user must decide whether the solution's readability is sufficient or not, and must be ready to influence the learning process when necessary. NEFCLASS must be seen as a tool that supports users in finding readable fuzzy classifiers. It is not an automatic classifier creator where data is fed in and a solution pops out. It is necessary that the user works with the tool (with work being the operative word).

For this reason it is necessary that NEFCLASS uses only very fast learning strategies to give users the possibility to interact with the tool. Therefore NEFCLASS does not induce fuzzy classification rules by searching for clusters, but by selecting rules from a grid (as shown later in section 4). By specifying initial fuzzy sets for each feature, the data space is structured, i.e. a certain granularity is provided for the feature space. Rules are found in this structure by finding places that contain data. Afterwards the fuzzy sets are tuned to enhance the performance of the rule base. The learning algorithm operates only on single dimensions, and it does not need to handle the high-dimensional feature space simultaneously. It is therefore a much simpler and faster approach, such as a cluster analysis approach, that creates fuzzy rules from data by searching for hyper-ellipsoidal [20] or hyper-rectangular clusters [21, 22].

In fuzzy clustering approaches [23, 24] fuzzy rules are constructed by projecting clusters on to the single dimensions [20]. In the case of hyper-ellipsoidal clusters, the projection causes a loss of information (see Fig 1), which is avoided when hyper-rectangular clusters are used [25]. However, in both cases the induced fuzzy rules are sometimes not easy to interpret. NEFCLASS modifies the fuzzy sets directly, and can therefore use constraints to make sure that the induced fuzzy rules can be interpreted well.

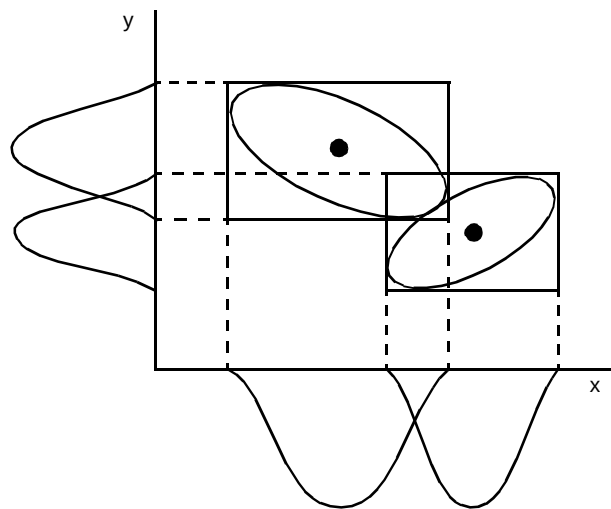


Fig 1 If fuzzy rules are created by projecting ellipsoidal clusters, a loss of information occurs — the resulting rule is only equivalent to an encompassing hyper-box.

3. The NEFCLASS model

A NEFCLASS model is a fuzzy classifier represented as a 3-layer feedforward architecture that is derived from a generic fuzzy perceptron [7] (see Fig 2).

The first layer U_1 contains the input units representing the pattern features. The activation a_x of a unit $x \in U_1$ is usually equal to its external input. However, it can be different if the input unit does some kind of preprocessing (normalisation, etc). The hidden layer U_2 holds k rule units representing k fuzzy rules, and the third layer U_3 consists of output units, one for each class. The activation of a rule unit $R \in U_2$, and the activation of an output unit $c \in U_3$ when a pattern $\mathbf{p}$ is propagated, are computed by:

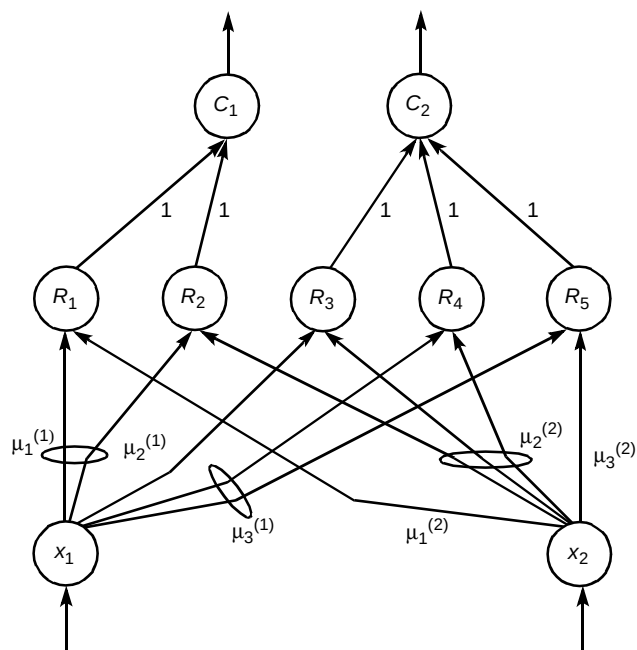


Fig 2 The architecture of the NEFCLASS model.

$$a_R^{(p)} = \min_{x \in U_1} \left\{ W(x, R) \left(a_x^{(p)} \right) \right\}$$

$$a_c^{(p)} = \max_{R \in U_2} \left\{ a_R^{(p)} \right\}$$

$$\text{or alternatively: } a_c^{(p)} = \sum_{\{R | R \in U_2 \wedge W(R, c) > 0\}} \frac{W(R, c) \cdot a_R^{(p)}}{W(R, c)},$$

where $W(x, R)$ is the fuzzy weight on the connection from input unit x to rule unit R . $W(R, c)$ is the weight on the connection from rule unit R to output unit c . For semantical reasons all these weights are fixed at 1, if a connection exists, or at 0, if no connection exists [18].

Alternatively, the output activation can be computed by a weighted sum instead of a maximum operation. This can be selected in accordance to the application problem. Instead of min and max, other t -norms and t -conorms can be used.

The rule base is an approximation of an (unknown) function $\phi: \mathbb{R}^n \rightarrow \{0,1\}^m$ that represents the classification task where $\phi(\mathbf{x}) = (c_1, \dots, c_m)$ such that $c_i = 1$ and $c_j = 0$ ($j \in \{1, \dots, m\}, j \neq i$), i.e. $\mathbf{x}$ belongs to class C_i . Because of the mathematics involved the rule base actually does not approximate ϕ but the function $\phi': \mathbb{R}^n \rightarrow \{0,1\}^m$. We will get $\phi(\mathbf{x})$ by $\phi(\mathbf{x}) = \psi(\phi'(\mathbf{x}))$, where ψ reflects the interpretation of the classification result obtained from a NEFCLASS system. In our case we will map the highest component of each vector $\mathbf{c}$ to 1 and its other components to 0 to determine the most likely class of a pattern (winner takes all). The information provided by the output vector can be additionally used to indicate overlapping of classes, or to reject certain classifications, for example, if a pattern is mapped to two classes with equal degrees of membership.

Representing a fuzzy classifier, the feedforward network has some advantages. For example, it is obvious that NEFCLASS uses shared weights on some of the connections (in Fig 2 this is shown by ellipses drawn around the connections). This way we make sure that for each linguistic value (e.g. ' x_1 is positive big') there is only one representation as a fuzzy set. It cannot happen that two fuzzy sets that are identical at the beginning of the learning process develop differently, and so the semantics of the rule base encoded in the structure of the network is not affected [17]. Connections that share a weight always come from the same input unit. In Fig 2 the hidden nodes R_2 and R_4 represent the rules:

R_2 : if x_1 is $\mu_2^{(1)}$ and x_2 is $\mu_2^{(2)}$ then pattern belongs to class C_1

R_4 : if x_1 is $\mu_3^{(1)}$ and x_2 is $\mu_2^{(2)}$ then pattern belongs to class C_2

Both rules use ' x_2 is $\mu_2^{(2)}$ ' in their antecedents. NEFCLASS makes sure that the representation of the fuzzy set $\mu_2^{(2)}$ is identical in both rules. The graph representation of the classifier visualises the data flow through the model, both for learning (backward path) and classification (forward path). It is easier to compare NEFCLASS to other fuzzy classification approaches, when the representation as a feedforward network is chosen.

This is the reason that graphical representations of neuro-fuzzy approaches still predominate in the literature. However, it should be noted that this kind of representation is not a prerequisite for applying a learning procedure, it is merely a convenience.

The fuzzy sets and the linguistic rules which perform the classification, and define the structure of the resulting NEFCLASS model, are obtained from a set of examples by learning. The following section describes the necessary algorithms.

4. Learning in NEFCLASS

A NEFCLASS system can be built from partial knowledge about the patterns, and can then be refined by learning, or it can be created from scratch by learning. A user has to define a number of initial fuzzy sets partitioning the domains of the input features, and must specify a value for k , i.e. the maximum number of rule nodes that may be created in the hidden layer.

The learning algorithm for NEFCLASS is described below. We assume that triangular membership functions are used that are given by three parameters:

$$\mu: \mathbb{R} \rightarrow [0, 1], \quad \mu(x) = \begin{cases} \frac{x-a}{b-a} & \text{if } x \in [a, b) \\ \frac{c-x}{c-b} & \text{if } x \in [b, c) \\ 0 & \text{otherwise} \end{cases}$$

In addition, the leftmost and rightmost membership functions for each variable are shouldered. Figure 3 shows a typical fuzzy partition used by NEFCLASS. The learning algorithm does not depend on triangular membership functions. It can easily be extended to other forms like trapezoidal or bell-shaped functions.

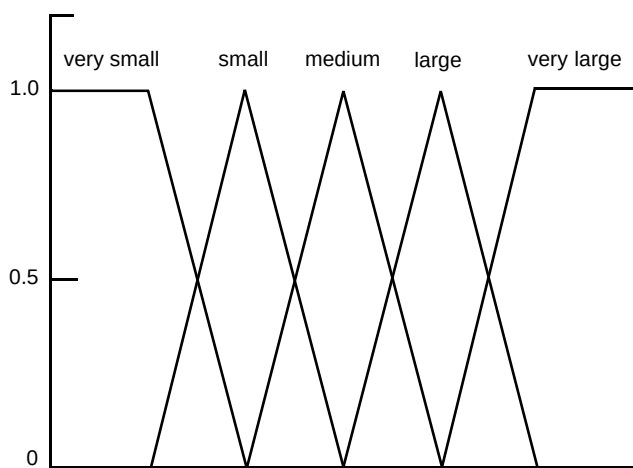


Fig 3 A typical fuzzy partition used by NEFCLASS.

The learning algorithm of NEFCLASS first creates a rule base from scratch by processing the training data set once. We will not go into details here, but provide only a short form of the training procedure. The complete learning algorithm can be found in Nauck et al [7]. The predefined membership functions of the input variables create a grid structure of overlapping hyperboxes in the data space. Each hyperbox is a potential fuzzy rule. The learning algorithm performs the following steps.

- Select the best k fuzzy rules from the grid structure. This is done by selecting each hyperbox that contains data and ranking them according to their classification performance. The user specifies a value for k , i.e. the maximum number of rules NEFCLASS is allowed to insert into the rule base.

- Iteratively process all patterns of the training set and use the classification error in a backpropagation-like algorithm to update the parameters of the membership functions. Depending on the demands of the user, the modifications must meet one or more of several constraints, for example:

— a membership function must not pass one of its neighbours,

— membership functions must be symmetrical,

— degrees of membership must add up to 1 for each value (i.e. membership functions intersect at a degree of membership of 0.5).

The learning algorithm is visualised in Fig 4. Figure 4(a) shows how rules are selected from a grid structure in feature space that is given by the fuzzy sets of the individual variables. In this case we allowed the system to create only three rules, therefore there are unclassified patterns. Figure 4(b) displays the situation after the learning algorithm for the membership functions was applied to improve the classification result. Here no constraints were used to restrict the learning process. As can be seen, the resulting fuzzy partitions are no longer nicely interpretable. Such a result is an indication that the learning process should be repeated with other parameters. In this case it would have been better to allow the system to create four rules and to use constraints for training the membership functions. This example is to illustrate that it is important for the user to work interactively with approaches like NEFCLASS to obtain readable solutions.

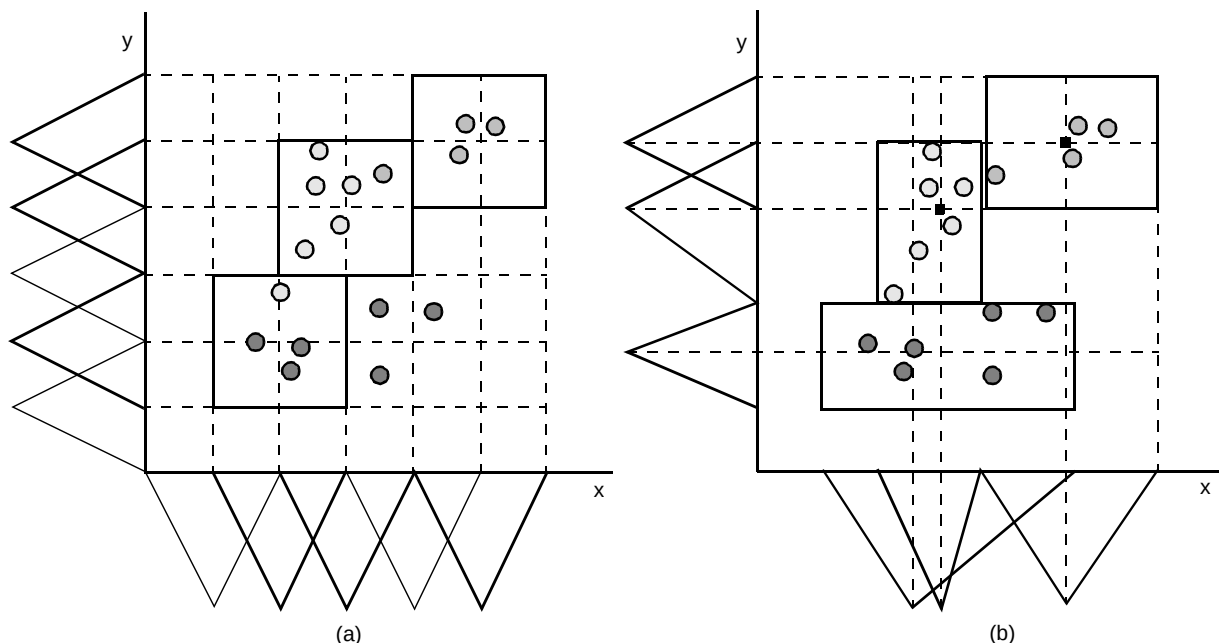


Fig 4 NEFCLASS selects rules from a grid structure (a) and modifies (in this case unconstrained) membership functions to improve the classification result (b).

The goal of applying NEFCLASS is to find a readable classifier. However, a good interpretation of the learning result cannot always be guaranteed, especially for high-dimensional problems. Because interpretation is one reason for using a fuzzy classifier in the first place, there is a need to enhance the learning algorithms of a neuro-fuzzy system with techniques for simplifying the obtained result.

With the pruning techniques available in our software NEFCLASS-X (see section 5) a learned rule base can be further improved. The rule editor of NEFCLASS-X supports the user with the rule simplification process. It also checks the rule base for consistency after each pruning step, and tells the user whether inconsistencies have occurred. An example where the rule editor has detected an error in a rule base is shown in Fig 5. The user can decide what actions have to be taken in case of an inconsistency, for example which rule to delete, if there are two rules with identical antecedents, but different conclusions.

NEFCLASS offers five pruning strategies.

- Evaluate rules

The rule learning procedure of NEFCLASS is invoked again, and only the best k rules are kept. The user can specify a (new) value for k . This strategy is useful, if the learning process was started with a large number of rules, and many of them have a poor performance measure.

- Delete inputs

NEFCLASS computes the correlations of the input variables with the class information, and offers the possibility to delete input variables that have a low correlation to the output. The user can specify the absolute value for the minimum correlation. The deletion of inputs can lead to inconsistencies in the rule base. The rule editor will notify the user of this and provide support in solving such problems.

- Delete rules

Rules that are only responsible for a few classifications can be deleted from the rule base, if they are not needed to cover assumptions in the data. The user can specify a percentage value that has to be reached by each rule to remain in the rule base.

- Delete antecedents

For each rule the antecedents that are not identical to the rule's degree of fulfilment in at least $z\%$, are deleted from the rule. The user can specify a value for z . This method can lead to an inconsistent rule base, which has to be repaired in the rule editor.

- Delete fuzzy sets

If a variable is partitioned by more than two fuzzy sets, sometimes the support for one or more of them can become quite large during learning. This can be seen as evidence that such a fuzzy set is superfluous. The user can specify a percentage value, and all fuzzy sets that cover more percent of the domain are deleted, which

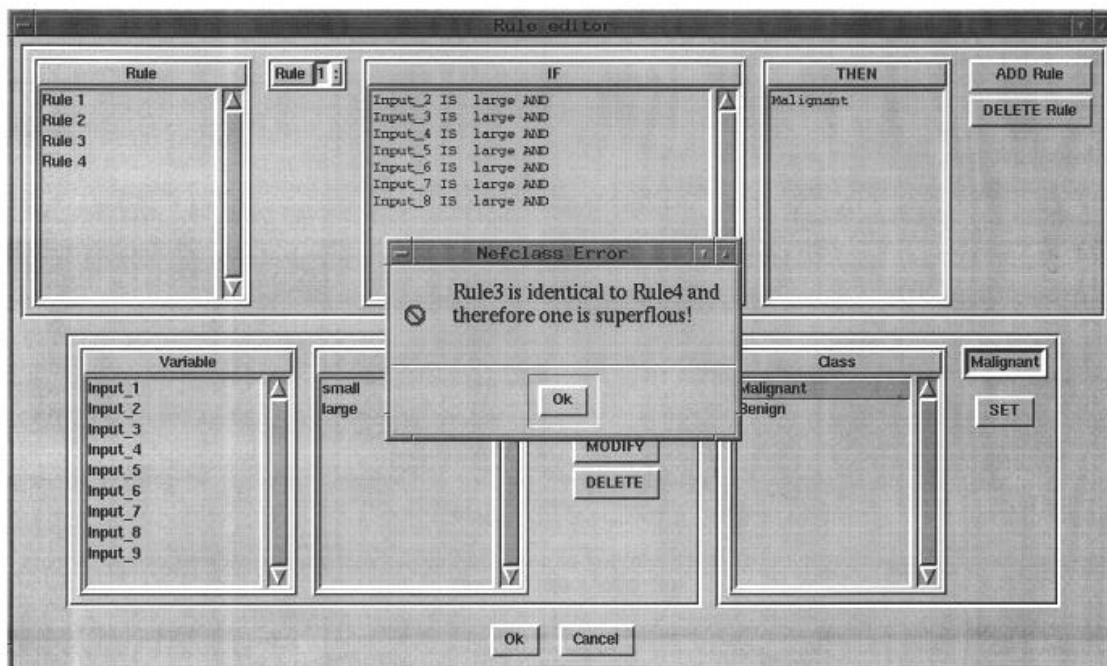


Fig 5 The rule editor of NEFCLASS-X allows to enter and to modify rules, and it checks the rule base for consistency.

leads to a reduction of variables in the antecedents. This procedure can also lead to an inconsistent rule base, which will be noticed by the rule editor.

Each of the five above-mentioned pruning strategies is applied interactively by the user. It is possible to view the evaluations of the pruning strategy which shall be applied prior to using it. In this way, the user can decide on the appropriate parameter for the pruning method. After the pruning method has been applied, the rule editor is invoked and displays the new rule base. If pruning results in inconsistencies or other problems such as identical rules, the user can solve these step by step. If the new rule base is not desirable, the pruning step can be cancelled. If the classification performance decreases after a pruning step, the learning process can be invoked to improve the classifier again, if possible. By saving and restoring NEFCLASS systems the user can try different strategies to improve the rule base until a set of rules is found that yields acceptable interpretability and performance.

5. Using NEFCLASS-X

This section shows how NEFCLASS-X can be used to obtain a classifier from data by using a small example — the iris data set [26]. The iris data is a well-known benchmark for classification approaches, although it represents a very simple classification problem. The data set consists of 150 patterns with 4 features. There are 3 classes (iris setosa, iris virginica and iris versicolour) that each contain 50 patterns. The first and second class are linearly separable to each other, and the second and third class slightly overlap. For training and testing the data was split into two sets that each contain 75 patterns (25 in each class). The application of NEFCLASS to another data set is described in Nauck et al [7].

NEFCLASS-X consists of a C++ program that implements the NEFCLASS approach and a separate user interface written in Tcl/Tk. The software was written for Unix environments, but it is possible to run it under Windows since Tcl/Tk is available for these platforms ¹.

We begin the training process to find a classifier for the iris data with loading the training set and allowing NEFCLASS-X to create 7 rules, where each variable uses 3 fuzzy sets. With these parameter settings there are $3^4 = 81$ possible rules. NEFCLASS-X finds 19 different rules within the data. Because we chose 'best' rule learning, NEFCLASS-X is forced to select only the best 7 rules. For training the membership functions we impose the constraints that no fuzzy sets must pass a neighbour and that the membership functions must intersect at a degree of membership of 0.5. The learning process stops if the error cannot be decreased for 10 consecutive epochs, and the best

classifier that is found during learning is presented to the user. After 34 epochs learning stops with 4 errors on the training set and 7 errors on the test set. This is not yet a very good result for the iris data set. In addition, the membership functions that are displayed by the graphical surface of NEFCLASS-X did not develop very well for the second variable (see Fig 6).

The next step is to try to simplify the classifier. Using the rule editor of NEFCLASS-X we can see that rules 1 to 4 are only responsible for the classification of a few per cent of all patterns (see Table 1).

Table 1 Classification performance of the rules found in the first training stage.

Rule	Class	Percentage of patterns classified
1	1	4.00
2	2	1.33
3	3	0.00
4	3	1.33
5	3	28.00
6	2	32.00
7	1	33.33

After pruning these 4 rules from the classifier, we obtain 9 errors on the training set and 7 errors on the test set. This means the performance decreased. Therefore we restart the learning process for the membership functions. After 55 epochs learning stops with 8 errors on the training set and 5 errors on the test set. Now we try to remove some variables from the classifier. NEFCLASS-X can show the correlations of the feature variables to the class information. For the iris data we find that the second feature has the lowest correlation (see Table 2). Therefore we prune this feature from the classifier.

Table 2 Correlations between the iris features and the class information.

Feature	Correlation
1	0.74
2	-0.48
3	0.95
4	0.96

After pruning the second feature, we obtain 7 errors on the training set and 5 errors on the test set. This means that we lost one error and the feature with the least interpretable fuzzy partition. Training cannot improve the error count. After inspecting the classifier again, we find that the first feature, that now has the smallest correlation with the

¹The source code of the software can be obtained from our WWW server at <http://fuzzy.cs.uni-magdeburg.de>.

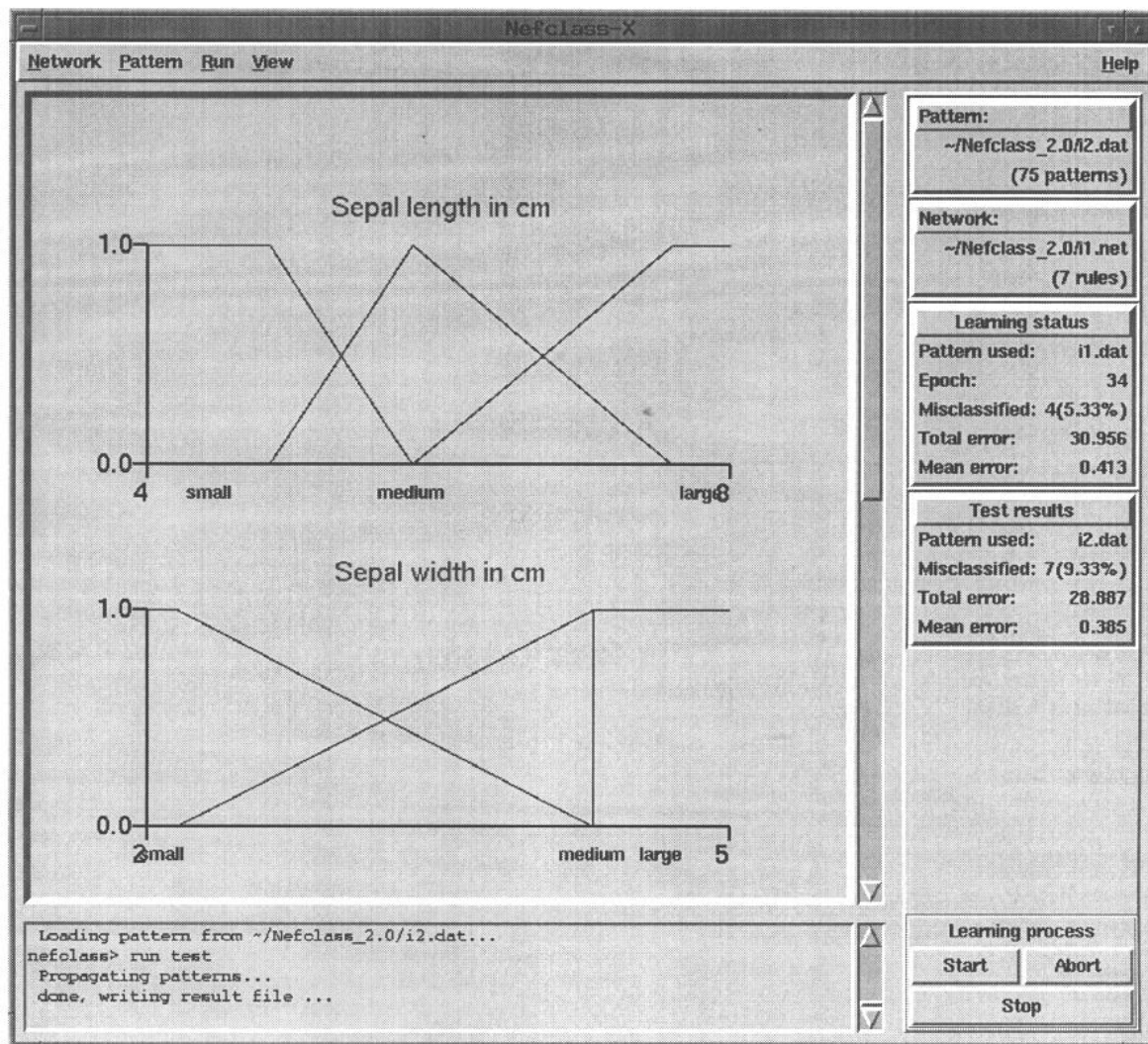


Fig 6 The surface of NEFCLASS-X displaying the membership functions after the first training stage — further steps are necessary to improve the readability (see section 5).

output, is responsible for the degree of fulfilment of all rules in most of the cases. This means that this feature is very often responsible for the result of the conjunction (minimum) in the rules' antecedents. Because the third and fourth variable seem to be better features considering the correlations, we prune the first feature from the classifier. After this we obtain 4 errors on the training set and 3 errors on the test set, a result, that cannot be further improved by training the membership functions.

However, it is possible that the classifier was driven into a local minimum during pruning, from where it cannot escape by training the membership functions. By interactive pruning we found that a classifier with three rules using only the third and fourth features seems to be the most promising. Therefore we create a new NEFCLASS model using now only the last two features. We let NEFCLASS create 3 rules by 'best per class' learning (9 rules are possible, 5 rules are found in the data, the best rule for each class is selected). After only 19 epochs of training the

membership functions we obtain a result with 3 errors on each data set. The rules are given in Table 3 and the fuzzy sets for the two remaining features are shown in Fig 7. We obtain the same performance if we create a classifier with training and test set exchanged (twofold cross validation). The resulting classifier is small and the interpretation of the fuzzy sets as small, medium and large is acceptable.

Table 3 Fuzzy rules to classify the iris data set.

R_1	if petal length is	small	and petal width is	small	then Iris-Setosa
R_2	if petal length is	medium	and petal width is	medium	then Iris-Versicolour
R_3	if petal length is	large	and petal width is	large	then Iris-Virginica

The example in this section shows how NEFCLASS-X can be used as an interactive tool to find a readable fuzzy

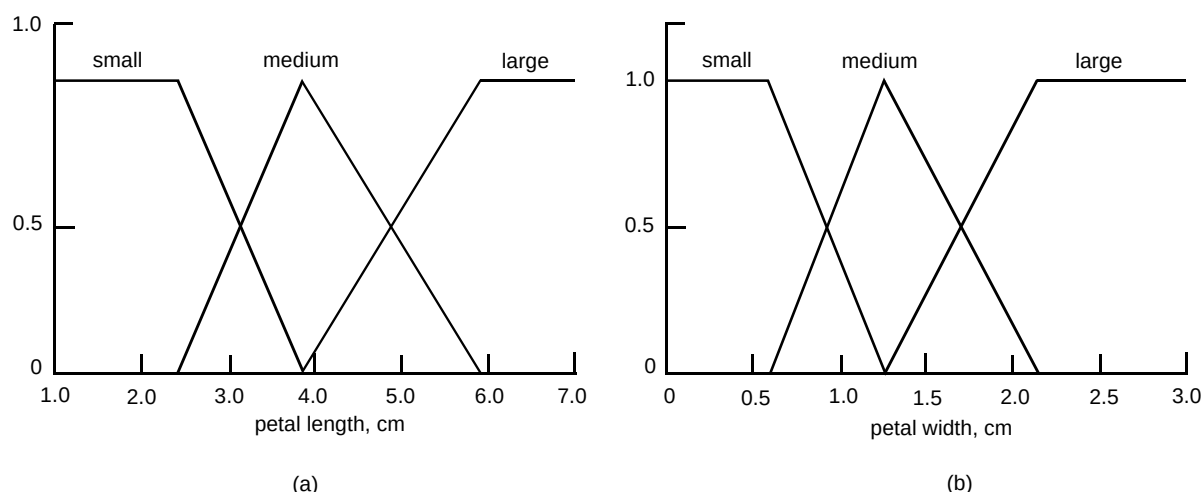


Fig 7 Membership functions for the final rule base of the iris classifier (third and fourth feature).

classifier. It is important to note that the task of creating an interpretable solution cannot be done without the user. The user must decide if the obtained classifier is sufficient for the purpose. While working with the tool, the user gains some knowledge about the data. This knowledge can be used to restart the training process to obtain more suitable solutions — classifiers that are either more readable or more accurate — depending on the demands of the user.

If the constraint that the membership function must intersect at degree 0.5 is dropped, one can obtain a good classification result (3 errors on training set and 2 errors on test set) right away (using all 4 variables and 7 rules, for example). However, the resulting fuzzy sets overlap strongly, and the classifier cannot be nicely interpreted, but it is a little bit more accurate. Users must decide what is more important, and how much work they are willing to put into creating a readable classifier.

6. Application of NEFCLASS

NEFCLASS can be applied in every domain where a classifier is needed, and where certain properties of the classifier are of interest:

- readability (instead of a black box model),
- simplicity and tolerance for vagueness (instead of a very accurate but complex model),
- low cost (instead of a long identification process for complex models).

The problem that is to be solved by NEFCLASS should have the following characteristics:

- direct dependency between input variables and classification, i.e. it is sufficient to assume independency between the inputs,
- there is no deep knowledge about the distribution of the data or the dependency of the variables,

- the problem is of low dimensionality (approximately less than 20 input variables),
- it is more important to obtain a rough solution fast, than to find a very accurate solution in a long process.

For medical problems, symptoms or test results can be used to classify patients. Especially for medical applications, it is important to stress that there is usually no desire for automatic classification. It is more important to support the decision-making process of the physician and to provide further information. A classifier built with NEFCLASS can do this. The physician can easily check the classifier for plausibility, and can verify why a certain classification result was obtained for a certain patient, by checking the degree of fulfilment of the individual fuzzy rules. Instead of a simple classification the physician also gets a description of the patient in terms of the fuzzy rules that are active for this patient. An example in the area of breast cancer diagnosis is discussed elsewhere [27, 28].

In stock market analysis there is often a lot of vague expert knowledge available from brokers. NEFCLASS is especially suitable in environments where prior knowledge exists, because such knowledge can be conveniently integrated into the classifier before training. This is usually not the case for other classification methods like statistics, neural networks, etc. In Siekmann et al [29] another neuro-fuzzy model researched by our group in co-operation with Siemens AG is described. It was applied to the prediction of the daily German stock market index, Dax, and refined prior expert knowledge by training. The applied neuro-fuzzy model differs from NEFCLASS in using bell-shaped fuzzy sets, and its architecture was especially designed to be implemented in the neural network simulation environment, SENN.

For insurance companies, the classification of customers is very important, e.g. to realise cross-selling strategies or to do risk assessment. In an ongoing industrial project we apply NEFCLASS to determine customers that are about to

cancel their contract and customers who should be offered new contracts. In this setting we are also researching the possibility of building meta-models with NEFCLASS.

Insurance companies often send customer records to local representatives in order to contact the customers and offer them additional contracts. To reduce the costs involved in this process it is important to identify customers very accurately. However, the sales personnel like to know why certain selected people are considered to be promising new customers. If there are a lot of misclassified records, the sales personnel tend to generalise these errors to all selected records and lose motivation to make any further contact with the selected persons. If the sales personnel also receive a simple linguistic description why the records they obtained were selected, they can check the description for plausibility and can verify the customer data against it. If a customer record fits the linguistic description only to some extent, this can be used as an explanation for an unsuccessful contact to the respective customer. The sales person also has the possibility to rank the customer records according to their correspondence with the model, in order to contact the most promising customers first.

NEFCLASS can be used to provide a simple linguistic description of the classification result, but another more accurate classifier may be used to actually process the customer data, if the performance of the simple descriptive model obtained by NEFCLASS turns out to be insufficient.

In this case, a second less accurate but more readable classifier obtains by NEFCLASS functions a (simplified) meta-model for the first classifier which is actually used as a model for the customer data. The first (very accurate) classifier can be realised by any approach, e.g. statistics or neural networks — but of course it can also be realised with NEFCLASS by using more membership functions and more rules to obtain a more accurate (but less readable) classifier.

7. Conclusions

We have discussed our neuro-fuzzy model NEFCLASS and its implementation NEFCLASS-X. The tool supports a user in obtaining a readable fuzzy classifier which learns from data. In addition to the learning strategies, the tool allows a rule base to be simplified by several interactive pruning methods. The example that we discussed above shows that a neuro-fuzzy approach is mainly suitable as a development tool. It cannot do all the work — the user still has to make an effort to obtain a sufficiently interpretable solution. An accurate and interpretable fuzzy classifier can usually not simply be found by an automatic learning process. The price for readability is a loss in accuracy.

The learning and pruning strategies of NEFCLASS are simple and fast heuristics, but they are not supposed to

function as an automatic process to create a fuzzy system. The user must be able to supervise and interpret the learning procedure in all its stages. The NEFCLASS software provides this functionality. The software tool and the data that was used by us to obtain the results presented in this paper, can be found on our WWW server fuzzy.cs.uni-magdeburg.de.

References

- 1 Zadeh L A: 'The roles of fuzzy logic and soft computing in the conception, design and deployment of intelligent systems', *BT Technol J*, **14**, No 4, pp 32—36 (October 1996).
- 2 Berenji H R and Khedkar P: 'Learning and tuning fuzzy logic controllers through reinforcements', *IEEE Trans Neural Networks*, **3**, pp 728—740 (September 1992).
- 3 Buckley J J and Hayashi Y: 'Fuzzy neural networks: a survey', *Fuzzy Sets and Systems*, **66**, pp 1—13 (1994).
- 4 Buckley J J and Hayashi Y: 'Neural networks for fuzzy systems', *Fuzzy Sets and Systems*, **71**, pp 265—276 (1995).
- 5 Halgamuge S K and Glesner M: 'Neural networks in designing fuzzy systems for real world applications', *Fuzzy Sets and Systems*, **65**, pp 1—12 (1994).
- 6 Jang J S R: 'ANFIS: adaptive-network-based fuzzy interference systems', *IEEE Trans Systems, Man and Cybernetics*, **23**, pp 665—685 (1993).
- 7 Nauck D, Klawonn F and Kruse R: 'Foundations of neuro-fuzzy systems', Wiley, Chichester (1997).
- 8 Tschichold-Guerman N: 'Generation and improvement of fuzzy classifiers with incremental learning using fuzzy rulenet', in George K M, Carrol J H, Deaton E, Oppenheim D and Hightower J (Eds): 'Applied computing', *Proc 1995 ACM Symposium on Applied Computing*, Nashville, pp 466—470, ACM Press, New York (February 1995).
- 9 Bezdek J C, Tsao E C K and Pal N R: 'Fuzzy Kohonen clustering networks', *Proc IEEE Int Conf on Fuzzy Systems*, pp 1035—1043, San Diego, CA (1992).
- 10 Vuorimaa P: 'Fuzzy self-organizing map', *Fuzzy Sets and Systems*, **66**, pp 223—231 (1994).
- 11 Kosko B: 'Neural networks and fuzzy systems — a dynamical systems approach to machine intelligence', Prentice Hall, Englewood Cliffs, NJ (1992).
- 12 Lin C T and Lee C S G: 'Neural fuzzy systems — a neuro-fuzzy synergism to intelligent systems', Prentice Hall, New York (1996).
- 13 Kruse R, Schwecke E and Heinsohn J: 'Uncertainty and vagueness in knowledge-based systems: numerical methods', Springer-Verlag, Berlin (1991).
- 14 Pearl J: 'Probabilistic reasoning in intelligent systems — networks of plausible inference', Morgan Kaufmann, San Francisco (1988).
- 15 Gebhardt J and Kruse R: 'Learning possibilistic networks from data', *Proc of Fifth Int Workshop on Artificial Intelligence and Statistics*, pp 233—244, Fort Lauderdale, Florida (1994).
- 16 Kruse R, Gebhardt J and Klawonn F: 'Foundations of fuzzy systems', Wiley, Chichester (1994).
- 17 Nauck D and Kruse R: 'A neuro-fuzzy method to learn fuzzy classification rules from data', *Fuzzy Sets and Systems*, **89**, pp 277—288 (1997).
- 18 Nauck D and Kruse R: 'How the learning of rule weights affects the interpretability of fuzzy systems', *Proc IEEE Int Conf on Fuzzy Systems*, pp 1235—1240, Anchorage (May 1998).

- 19 Bersini H and Bontempi G: 'Now comes the time to defuzzify neuro-fuzzy models', *Fuzzy Sets and Systems*, 90, pp 161—170 (1997).
- 20 Klawonn F and Kruse R: 'Constructing a fuzzy controller from data', *Fuzzy Sets and Systems*, 85, pp 177—193 (1997).
- 21 Abe S and Lan M S: 'A method for fuzzy rules extraction directly from numerical data and its application to pattern classification', *IEEE Trans Fuzzy Systems*, 3, No 1, pp 18—28 (February 1995).
- 22 Abe S, Lan M S and Thawonmas R: 'Tuning of a fuzzy classifier derived from data', *Int J Approximate Reasoning*, 14, No 1, pp 1—24 (January 1996).
- 23 Bezdek J C: 'Pattern recognition with fuzzy objective function algorithms', Plenum Press, New York (1981).
- 24 Bezdek J C and Pal S K (Eds): 'Fuzzy models for pattern recognition', IEEE Press, New York (1992).
- 25 Tshichold-Guerman N: 'The neural network model rulenet and its application to mobile robot navigation', *Fuzzy Sets and Systems*, 85, pp 287—303 (1997).
- 26 Fisher R A: 'The use of multiple measurements in taxonomic problems', *Annual Eugenics*, 7 (Part II), pp 179—188 (1936).
- 27 Nauck D and Kruse R: 'New learning strategies for NEFCLASS', *Proc Seventh International Fuzzy Systems Association World Congress IFSA'97*, IV, pp 50—55, Prague (1997).
- 28 Nauck D and Kruse R: 'What are neuro-fuzzy classifiers?', *Proc Seventh International Fuzzy Systems Association World Congress IFSA'97*, III, pp 228—233, Prague (1997).
- 29 Siekmann S, Kruse R, Neuneier R and Zimmermann H G: 'Advanced neuro-fuzzy techniques applied to the German stock index Dax', *Proc Second European Workshop on Fuzzy Decision Analysis and Neural Networks for Management, Planning and Optimization (EFDAN'97)*, pp 170—179, Dortmund (June 1997).



Detlef Nauck received the MS and Phd degrees in computer science from the University of Braunschweig, Germany in 1990 and 1994, respectively.

He was employed as a Researcher and Senior Researcher at the University of Braunschweig. He is currently a Senior Research Fellow and Senior Lecturer at the University of Magdeburg, Germany. His current research interests include neural networks, fuzzy systems, machine learning and knowledge discovery in databases. He is a Programme Co-Chair of the International

Conference on Neural Information Processing 1999 in Sydney.



Rudolf Kruse received the PhD degree and Venia Legendi in Mathematics from the University of Braunschweig, Germany in 1980 and 1984 respectively. From 1986-1996 he was Professor of Computer Science at the University of Braunschweig. Since 1996 he has been Professor of Computer Science at the University of Magdeburg, Germany.

His current research interests include neural networks, fuzzy systems, uncertainty management, data fusion, and knowledge discovery in databases. He is an active collaborator in several European research

projects on data fusion and uncertainty management.